

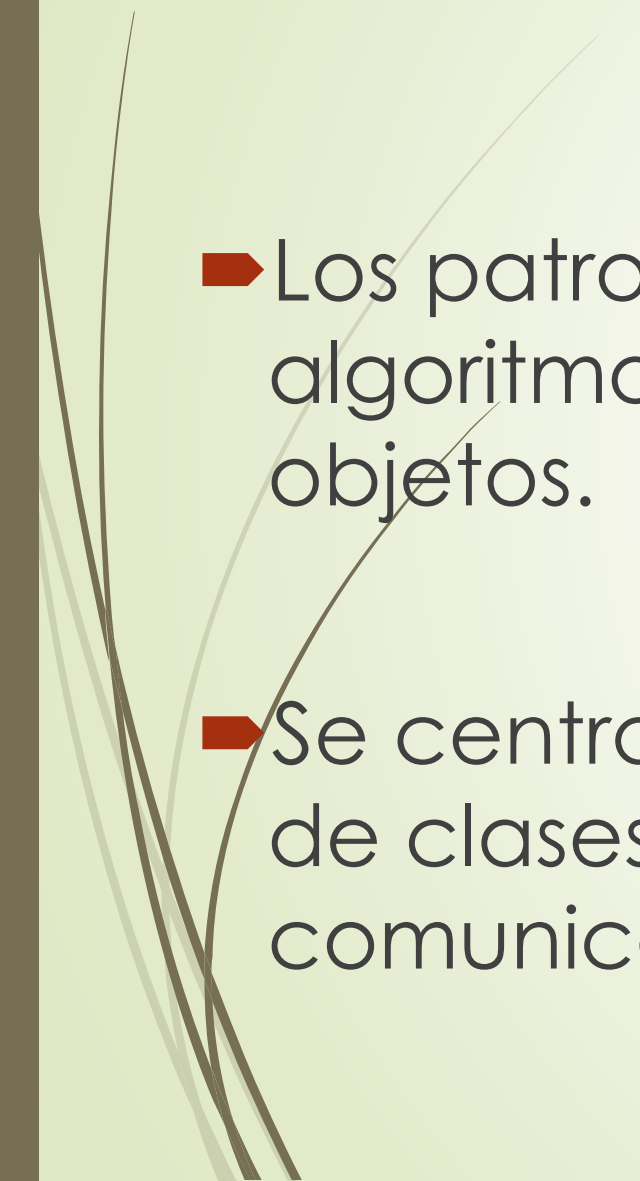


# PATRÓN STRATEGY

ING. ZOILA RUIZ, PhD



# INTRODUCCIÓN P. COMPORTAMIENTO

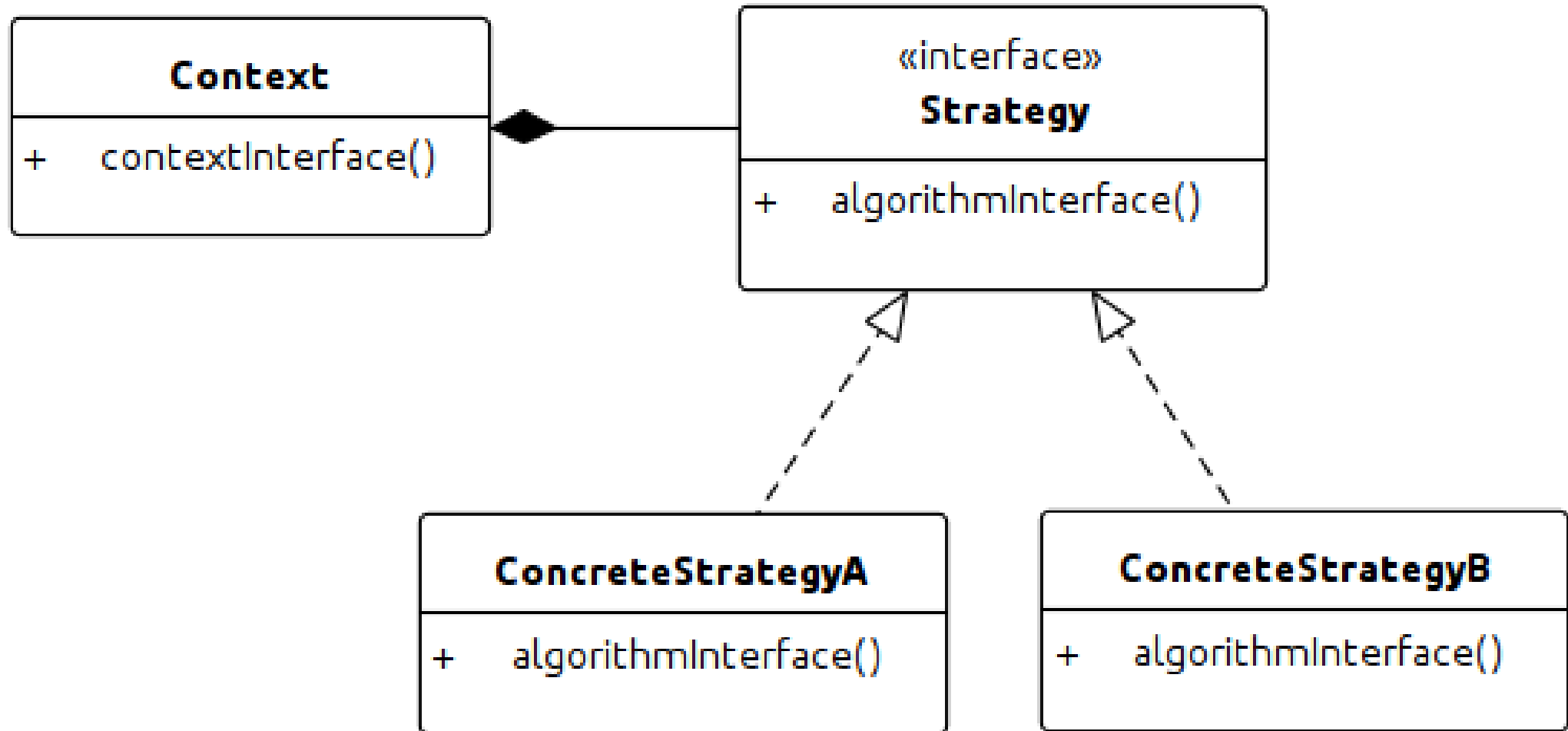
- Los patrones de comportamiento gestionan algoritmos, relaciones y responsabilidades entre objetos.
  - Se centran en la interacción entre asociaciones de clases y objetos definiendo cómo se comunican entre sí.
- 



# Patrón Strategy

- Determina cómo se debe realizar el intercambio de mensajes entre diferentes objetos para resolver una tarea.
- Crea un objeto que puede comportarse de formas diferentes.
- *Este patrón define un conjunto de algoritmos, encapsula cada uno de ellos y los hace intercambiables. Permite que el algoritmo pueda variar independientemente de los clientes que lo utilicen.*

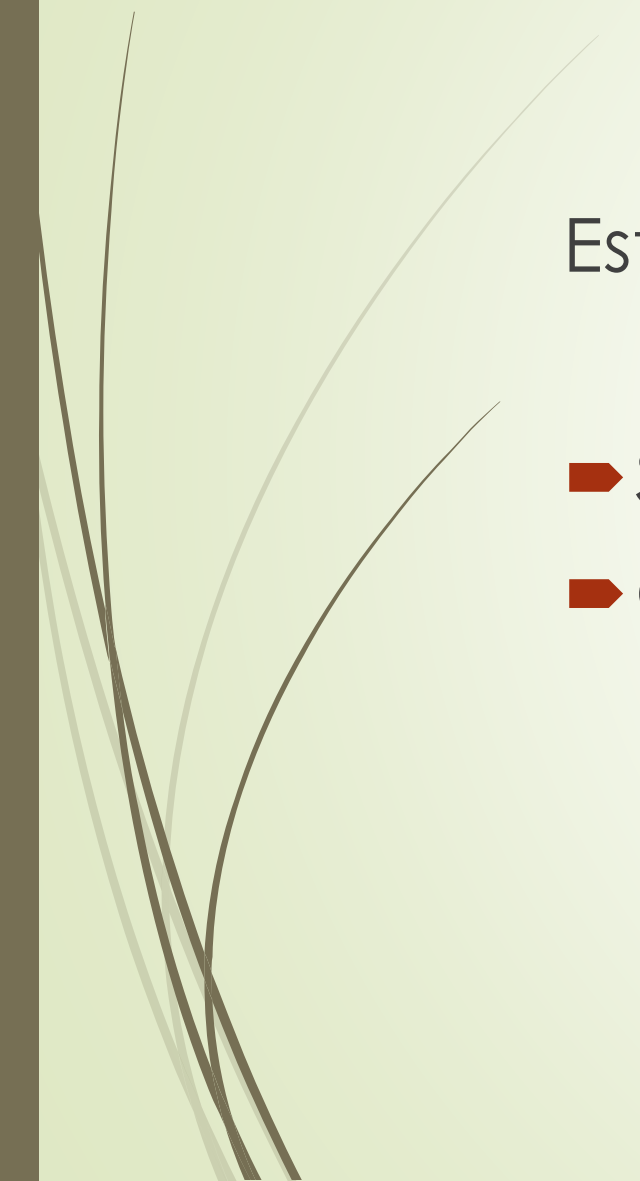
# Estructura del Patrón Strategy





# Estructura del Patrón Strategy

Este patrón cumple con dos principios de SOLID:

- Single responsibility
  - Open -closed
- 



# Estructura del Patrón Strategy

Se aplica cuando:

- Muchas clases relacionadas difieren sólo en su comportamiento
- Se necesita distintas variantes del mismo algoritmo
- Una clase define muchos comportamientos



# Estructura del Patrón Strategy

## Implementación

- Conviene analizar si es posible encapsular comportamiento común a todas las estrategias en una superclase.
- El cliente puede pasar la información necesaria al algoritmo o bien pasarse asimismo.
- El cliente puede evitar la creación innecesaria de objetos cuando la estrategia solicitada es idéntica a la última



# Estructura del Patrón Strategy

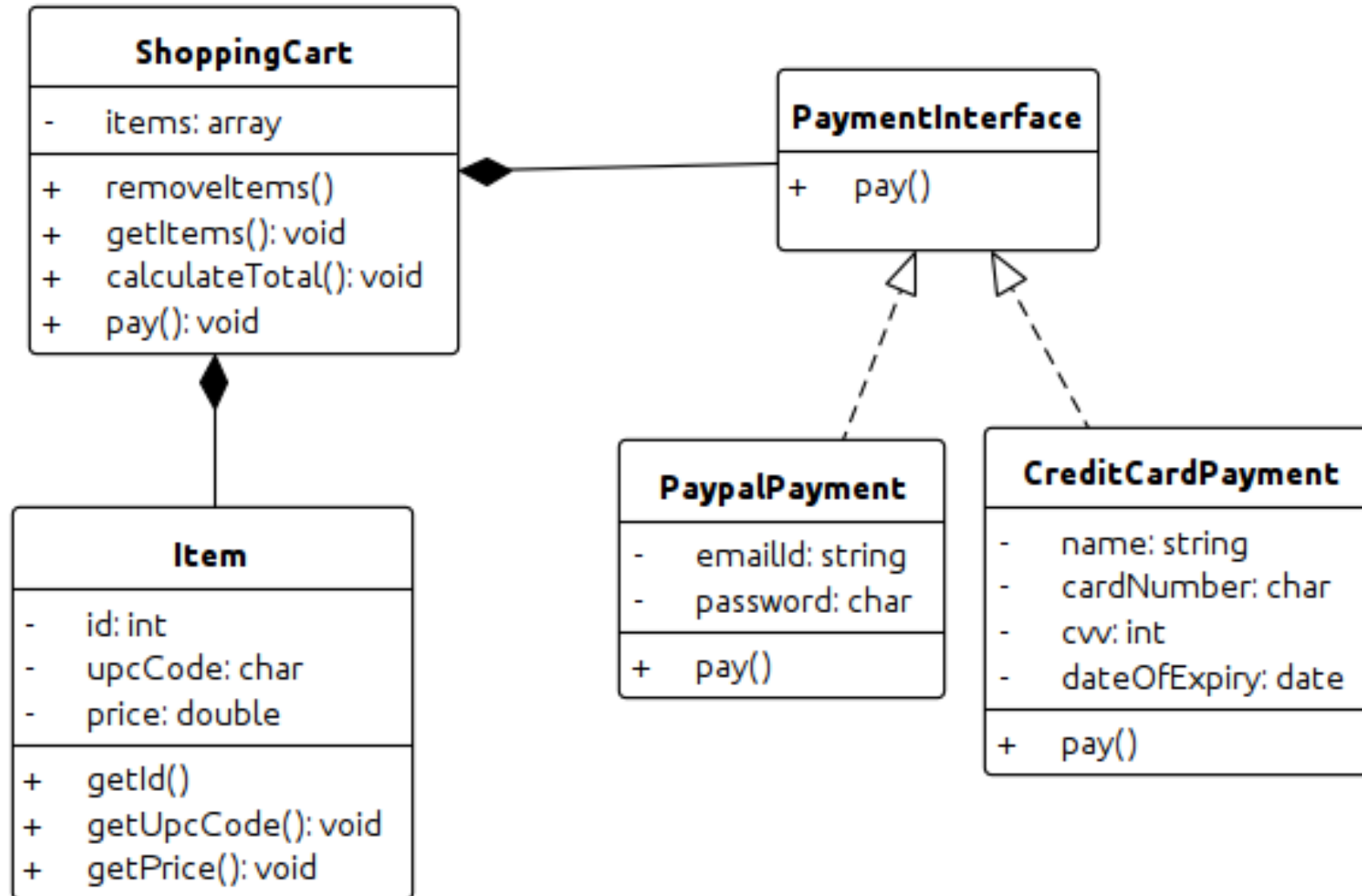
## Ventajas:

- Factoriza aspectos comunes de una familia de algoritmos y utilizarlos en las clases base de la jerarquía.
- El encapsulamiento ofrece una alternativa a la especialización por herencia del contexto.
- Sistematiza el uso de implementaciones alternativas.

## Desventajas

- Menor eficiencia, porque aumenta el número de objetos creados

# Ejemplo:



Ej

